

SIMATS ENGINEERING

Assignment

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1110

COURSE OUTCOME COVERED

CO1: *Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices.*

CO2: *Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system.*

CO3: *Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.*

Assignment Weightage: 100%

QUESTIONS: 1

Total Marks:100

Annexure A

Assignment

Code of Conduct:

I V. V. Naveen Kumar (192321018) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

vvnaveenkumar

Course Code & Name	CSA11 – Object Oriented Analysis and Design
Assignment Title	Object-Oriented Analysis and UML-Based Design of an AI-Enabled Smart Healthcare Appointment and Patient Support System (AI-SHAPS)
Course Outcome(s) – CO	CO1: Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices. CO2: Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system. CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.
Bloom's Taxonomy Level	BL3 – Apply, BL4 – Analyse, BL5 – Evaluate, BL6 – Create
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education SDG 9 – Industry, Innovation and Infrastructure SDG 11 – Sustainable Cities and Communities

Problem Statement

Analyse and develop an Object-Oriented Analysis and Design model for an AI-Enabled Smart Healthcare Appointment and Patient Support System (AI-SHAPS). The system shall integrate patients, doctors, reception staff, administrators, laboratory personnel, and pharmacy staff through a centralized intelligent platform. Patients shall register, manage profiles, search doctors, book/reschedule/cancel appointments, receive reminders, view prescriptions and laboratory reports, and submit service requests. Doctors shall manage availability, appointments, clinical notes, prescriptions, and follow-up requests. The system shall provide intelligent doctor/slot recommendations based on specialty, availability, patient preferences, and historical appointment patterns. Laboratory and pharmacy modules shall update report and prescription status. Working from this scenario, students shall identify actors, objects, classes, attributes, methods, and relationships and develop Use Case, Class, Sequence, Activity, State Transition, Package, Component, and Deployment Diagrams. Students shall evaluate object responsibilities and apply appropriate design principles and patterns to achieve modularity, maintainability, reusability, security, and scalability.

Constraints

The system shall clearly define the system boundary, actors, objects, classes, attributes, operations, and relationships. UML diagrams shall be consistent with one another, and the operations identified in the Class Diagram shall support the interactions represented in the Sequence and Activity Diagrams.

The design shall demonstrate appropriate use of encapsulation, abstraction, inheritance, polymorphism, cohesion, coupling, and responsibility assignment. Suitable design patterns shall be selected and justified based on the identified design problems.

The final model shall document assumptions and constraints and ensure that all UML models are complete, consistent, unambiguous, traceable, and feasible.

Tools Can Be Used

1. StarUML
2. Visual Paradigm
3. Draw.io
4. PlantUML

5. Papyrus Tools
6. ArgoUML
7. Enterprise Architect
8. Lucidchart

Assessment Rubrics

Criteria Mapping) (CO	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding, Object Identification & Requirements Analysis (CO1)	10	Clearly analyses the problem domain and accurately identifies all major actors, objects, system responsibilities, and requirements from the given scenario.	Identifies most actors, objects, and requirements with minor gaps or inaccuracies.	Identifies basic actors and objects but analysis is incomplete or contains some inconsistencies.	Poor understanding of the problem; major actors, objects, and requirements are missing or incorrectly identified.
Use Case Modelling & Actor Identification (CO2)	10	Correctly identifies all relevant actors and use cases; Use Case Diagram clearly represents system boundary, relationships, and interactions.	Most actors and use cases are correctly identified with minor modelling errors.	Basic actors and use cases are identified, but relationships or system boundary need improvement.	Actors and use cases are poorly identified; diagram is incomplete or inconsistent.
Class Identification, Attributes, Methods & Relationships (CO2/CO3)	15	Classes, attributes, methods, associations, aggregation/composition, inheritance, and dependencies are accurately identified and logically justified.	Class structure is mostly correct with minor errors in attributes, methods, or relationships.	Basic classes and relationships are identified but lack detail or contain several modelling issues.	Classes, attributes, methods, or relationships are missing, inappropriate, or poorly structured.
UML Behavioural & Interaction Modelling (CO2)	15	Sequence, Activity, and State Transition Diagrams accurately represent system behaviour and are fully consistent with the identified use cases and classes.	Behavioural diagrams are mostly correct with minor inconsistencies.	Basic behavioural diagrams are provided but some interactions, activities, or states are incomplete.	Behavioural diagrams are missing, unclear, or inconsistent with the proposed system.
UML Architectural Modelling & Design Evaluation (CO3)	15	Package, Component, and Deployment Diagrams clearly represent modular structure, system components, dependencies, and deployment environment with strong justification.	Architectural diagrams are mostly correct with minor omissions or modelling errors.	Basic architectural representation is provided but lacks clarity or sufficient justification.	Architectural diagrams are incomplete, inappropriate, or inconsistent with the system design.
Object Responsibilities, Design Principles, Axioms & Patterns (CO3)	15	Responsibilities are appropriately assigned; design axioms, cohesion, coupling, encapsulation, abstraction, and suitable design patterns are clearly justified and effectively applied.	Design principles and patterns are appropriately applied with minor limitations in justification.	Some design principles are identified, but application or justification is limited.	Design principles, axioms, responsibilities, or patterns are absent or incorrectly applied.
Consistency, Traceability & Overall Design Quality (CO3)	10	All UML models are mutually consistent, requirements are traceable to use cases/classes, and the overall design is complete, feasible, scalable, and maintainable.	Design is largely consistent and traceable with minor gaps.	Some consistency and traceability are demonstrated, but important links are missing.	Major inconsistencies exist between requirements, use cases, classes, and UML diagrams.
Reflection, SDG Relevance & Learning Outcomes	10	Provides insightful reflection on design decisions, challenges, OOAD concepts learned, and clearly establishes the relevance of the selected SDGs.	Provides good reflection with reasonable connection to design decisions and SDGs.	Reflection is present but generic with limited discussion of learning and SDG relevance.	Reflection is missing, superficial, or unrelated to the assignment and SDGs.
Total	100				

Deliverables

To receive full credit, each submission must include the following:

1. Problem Analysis and Requirement Summary
2. Actor Identification
3. Use Case Descriptions
4. Use Case Diagram
5. Object and Class Identification
6. Class Diagram
7. Sequence/Interaction Diagram(s)
8. Activity Diagram
9. State Transition Diagram
10. Package Diagram
11. Component Diagram
12. Deployment Diagram
13. Object Responsibility Analysis
14. Design Axioms/Principles and Design Pattern Justification
15. Assumptions and Constraints
16. Requirement-to-Use Case/Class Traceability
17. Implementation/Prototype and Results
18. Reflection on Design Decisions and Learning Outcomes
19. GitHub Upload

1. Problem Analysis and Requirement Summary

1.1 Problem Statement Summary

The AI-Enabled Smart Healthcare Appointment and Patient Support System (AI-SHAPS) is a centralized, intelligent platform that connects patients, doctors, receptionists, administrators, laboratory personnel, and pharmacy staff. The system is required to support end-to-end healthcare appointment management -- from patient registration and doctor discovery, through AI-assisted slot recommendation, appointment booking and rescheduling, to clinical documentation, laboratory reporting, prescription issuance, and pharmacy fulfilment -- while keeping every stakeholder informed through timely notifications.

1.2 Functional Requirements

- Patient registration, authentication, and profile management.
- Doctor search and filtering by specialty, availability, and patient preference.
- AI-based recommendation of suitable doctors and appointment slots using specialty, availability, preferences, and historical patterns.
- Appointment booking, rescheduling, and cancellation with real-time slot validation.
- Automated reminders and notifications to patients and doctors.
- Viewing of digital prescriptions and laboratory reports by patients.
- Submission and tracking of patient service requests.
- Doctor-side management of availability, clinical notes, prescriptions, and follow-up requests.
- Laboratory module to upload and update the status of diagnostic reports.
- Pharmacy module to update prescription fulfilment status.
- Administrative functions for user management, reporting, and system configuration.

1.3 Non-Functional Requirements

- Modularity and maintainability through loosely coupled, highly cohesive components.
- Scalability to support a growing number of concurrent patients, doctors, and appointments.
- Security of sensitive medical data (authentication, authorization, encrypted storage/transport).
- Reliability and consistency of appointment and clinical data across modules.
- Usability across web and mobile client interfaces.
- Traceability between requirements, use cases, classes, and behavioural models.

1.4 Requirement Summary Table

Req. ID	Requirement	Priority
REQ-01	Patient registration, login and profile management	High
REQ-02	Doctor search and AI-based recommendation	High
REQ-03	Appointment booking / reschedule / cancellation	High
REQ-04	Automated reminders and notifications	Medium
REQ-05	View prescriptions and lab reports	High
REQ-06	Patient service request submission and tracking	Medium
REQ-07	Doctor availability, notes, prescription management	High
REQ-08	Laboratory report upload and status update	High

Req. ID	Requirement	Priority
REQ-09	Pharmacy prescription status update	High
REQ-10	Administration: user management, reports, configuration	Medium

2. Actor Identification

Actors represent external entities (human or system) that interact with AI-SHAPS. The following primary and system actors were identified from the problem scenario.

Actor	Type	Description
Patient	Primary (Human)	Registers, manages profile, searches doctors, books/reschedules/cancels appointments, views prescriptions/lab reports, submits service requests.
Doctor	Primary (Human)	Manages availability, views appointments, records clinical notes, issues prescriptions, requests follow-ups.
Receptionist	Primary (Human)	Manages appointments on behalf of patients, registers walk-in patients, assists patients at the front desk.
Administrator	Primary (Human)	Manages users, generates system reports, configures system-wide settings.
Lab Personnel	Primary (Human)	Uploads laboratory reports and updates their status.
Pharmacy Staff	Primary (Human)	Updates prescription fulfilment status and dispenses medication.
AI Recommendation Engine	Secondary (System)	Analyses specialty, availability, preferences and history to recommend doctors/slots.
Notification System	Secondary (System)	Sends SMS/email/push reminders and confirmations to patients and doctors.

3. Use Case Descriptions

UC-01: Book Appointment

Use Case	UC-01: Book Appointment
Primary Actor	Patient
Preconditions	Patient is registered and logged in.
Main Flow	1. Patient searches for a doctor by specialty/symptom. 2. AI Recommendation Engine suggests suitable doctors and slots. 3. Patient selects a doctor and slot. 4. System validates slot availability. 5. System creates the appointment and sets status to Confirmed. 6. Notification System sends confirmation.
Postconditions	A new Appointment is created and linked to the Patient and Doctor.
Alternate / Exception Flow	If the slot is no longer available, the system shows alternate slots (extension of UC-01).

UC-02: Reschedule Appointment

Use Case	UC-02: Reschedule Appointment
Primary Actor	Patient
Preconditions	An existing Confirmed appointment exists.
Main Flow	1. Patient selects appointment to reschedule. 2. System shows available alternate slots (optionally AI-recommended). 3. Patient selects a new slot. 4. System updates appointment date/time and notifies doctor.
Postconditions	Appointment state changes to Rescheduled then Confirmed.
Alternate / Exception Flow	If no alternate slot is available, patient is prompted to cancel or wait-list.

UC-03: Cancel Appointment

Use Case	UC-03: Cancel Appointment
Primary Actor	Patient / Receptionist
Preconditions	An existing appointment exists.
Main Flow	1. Actor selects appointment. 2. System requests cancellation reason. 3. System updates status to Cancelled and frees the slot. 4. Notification sent to doctor and patient.
Postconditions	Appointment status becomes Cancelled; slot released.
Alternate / Exception Flow	If cancellation window has passed, a cancellation policy message is shown.

UC-04: Recommend Doctor / Slot

Use Case	UC-04: Recommend Doctor / Slot
Primary Actor	AI Recommendation Engine
Preconditions	Patient has submitted search criteria.

Main Flow	1. Engine analyses specialty match, doctor availability, patient preference and historical patterns. 2. Engine ranks doctors/slots. 3. Ranked list returned to Appointment Service / UI.
Postconditions	A ranked recommendation list is produced for the patient to choose from.
Alternate / Exception Flow	If no match is found, the engine broadens the criteria and retries.

UC-05: Manage Availability

Use Case	UC-05: Manage Availability
Primary Actor	Doctor
Preconditions	Doctor is logged in.
Main Flow	1. Doctor opens availability calendar. 2. Doctor adds/edits/removes available slots. 3. System persists updated schedule.
Postconditions	Doctor's Slot list is updated and visible to search/recommendation.
Alternate / Exception Flow	If a slot removed already has a booking, system warns before removal.

UC-06: Issue Prescription

Use Case	UC-06: Issue Prescription
Primary Actor	Doctor
Preconditions	An appointment/consultation is In-Progress or Completed.
Main Flow	1. Doctor records clinical notes. 2. Doctor adds medicines and dosage. 3. System generates Prescription linked to the Appointment. 4. Notification sent to Pharmacy module and patient.
Postconditions	A Prescription record is created with status Pending.
Alternate / Exception Flow	Doctor may request a follow-up appointment instead of / in addition to a prescription.

UC-07: Upload Lab Report

Use Case	UC-07: Upload Lab Report
Primary Actor	Lab Personnel
Preconditions	A lab test has been conducted for a patient.
Main Flow	1. Lab Personnel selects the patient/test. 2. Uploads the report file and result summary. 3. System sets report status to Available. 4. Notification sent to patient and doctor.
Postconditions	LabReport record created/updated and linked to patient/appointment.
Alternate / Exception Flow	If the report is incomplete, status remains Pending and a reminder is scheduled.

UC-08: Update Prescription Status

Use Case	UC-08: Update Prescription Status
-----------------	-----------------------------------

Primary Actor	Pharmacy Staff
Preconditions	A prescription exists with status Pending.
Main Flow	1. Pharmacy Staff verifies medicine availability. 2. Staff dispenses medication. 3. System updates prescription status to Dispensed.
Postconditions	Prescription status becomes Dispensed; patient notified.
Alternate / Exception Flow	If a medicine is unavailable, status is set to Partially Dispensed with remarks.

UC-09: Manage Users

Use Case	UC-09: Manage Users
Primary Actor	Administrator
Preconditions	Administrator is logged in with valid access level.
Main Flow	1. Administrator views the user list. 2. Administrator creates, edits, activates/deactivates user accounts (patients, doctors, staff). 3. Changes are persisted and audited.
Postconditions	User records are created/updated in the system.
Alternate / Exception Flow	Unauthorized attempts are blocked and logged.

UC-10: Submit Service Request

Use Case	UC-10: Submit Service Request
Primary Actor	Patient
Preconditions	Patient is logged in.
Main Flow	1. Patient selects request type (e.g., report correction, billing query). 2. Patient enters description. 3. System creates a ServiceRequest with status Open and routes it to the relevant staff.
Postconditions	A ServiceRequest is created and trackable by the patient.
Alternate / Exception Flow	If routed department is unavailable, request stays Open with an escalation flag.

4. Use Case Diagram

The diagram below shows the system boundary of AI-SHAPS, all primary human actors (Patient, Doctor, Receptionist, Administrator, Lab Personnel, Pharmacy Staff), the secondary system actor (AI Recommendation Engine), the identified use cases, and their associations. The <<include>> relationship shows that Book Appointment always invokes Recommend Doctor/Slot.



Figure 4.1: Use Case Diagram of AI-SHAPS

5. Object and Class Identification

Applying noun-phrase analysis and CRC-style reasoning to the problem statement, the following candidate classes, their category (entity/boundary/control), and key attributes/operations were identified.

Class	Category	Key Attributes	Key Operations
User (abstract)	Entity	userId, name, email, phone, password	register(), login(), updateProfile()
Patient	Entity	patientId, dob, medicalHistory, preferences	searchDoctor(), bookAppointment(), cancelAppointment()
Doctor	Entity	doctorId, specialty, qualification, availability	manageAvailability(), issuePrescription()
Receptionist	Entity	staffId, deskNo	manageAppointments(), registerWalkInPatient()
Administrator	Entity	adminId, accessLevel	manageUsers(), generateReports()
LabPersonnel	Entity	labStaffId, labSection	uploadLabReport(), updateReportStatus()
PharmacyStaff	Entity	pharmacyStaffId, counterNo	updatePrescriptionStatus(), dispenseMedication()
Appointment	Entity	appointmentId, dateTime, status, mode	confirm(), reschedule(), cancel(), complete()
Slot	Entity	slotId, startTime, endTime, isAvailable	-
Prescription	Entity	prescriptionId, medicines, dosage, status	generate(), updateStatus()
LabReport	Entity	reportId, testType, result, status	upload(), updateStatus()
ServiceRequest	Entity	requestId, type, description, status	submit(), track()
Notification	Control/Entity	notificationId, message, channel, timestamp	send()
AIRecommendationEngine	Control	engineId, modelVersion	recommendDoctor(), recommendSlot(), analyzeHistory()
AppointmentBookingUI	Boundary	-	renders search & booking screens

6. Class Diagram

The Class Diagram below models the static structure of AI-SHAPS: inheritance from the abstract User class to all human-actor roles, associations between Patient/Doctor and Appointment, aggregation between Doctor and Slot, and dependency relationships from AIRecommendationEngine and Notification to the core entities. Multiplicities are shown on each association.

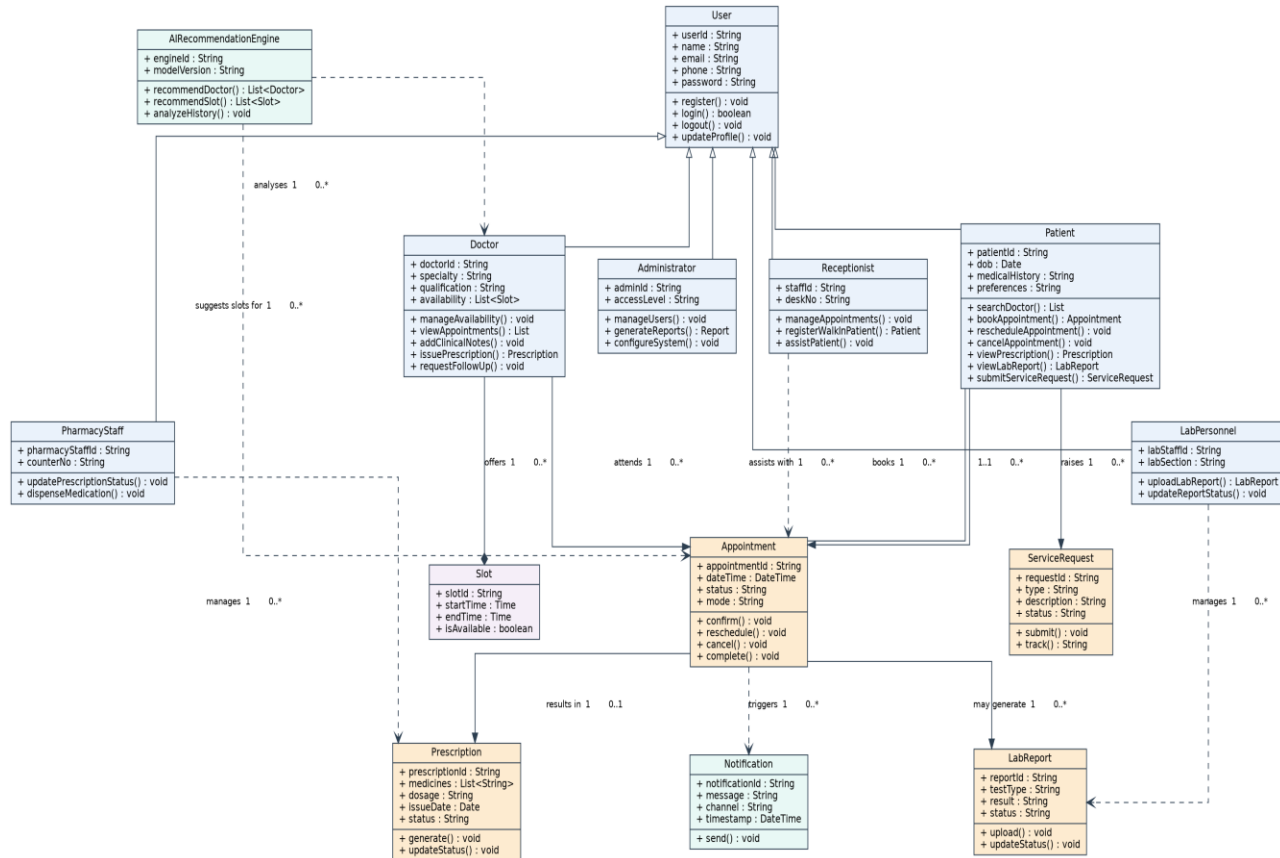


Figure 6.1: Class Diagram of AI-SHAPS

7. Sequence / Interaction Diagram

The sequence diagram below illustrates the 'Book Appointment' scenario end-to-end: the patient searches for a doctor, the AI Recommendation Engine ranks doctors/slots, the patient selects a slot, the Appointment Service reserves it, an Appointment object is created, and the Notification Service confirms the booking to the patient.

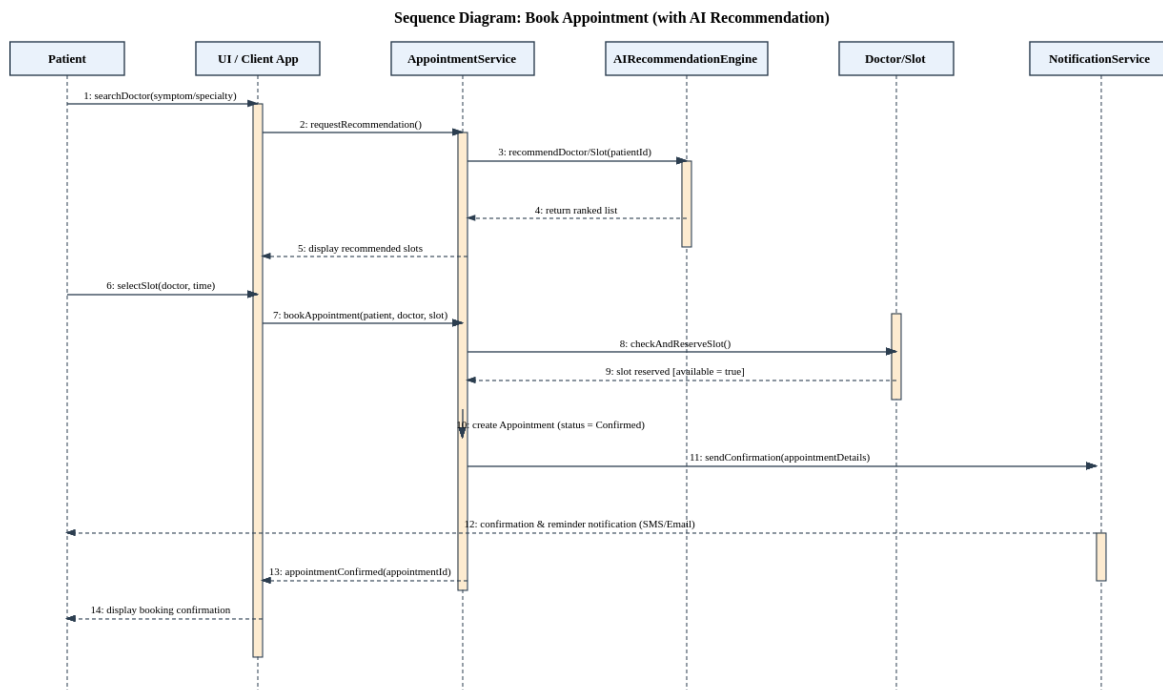


Figure 7.1: Sequence Diagram - Book Appointment

8. Activity Diagram

The activity diagram models the workflow of the appointment booking process, including the decision points for slot availability and optional payment, and continues through to consultation and clinical documentation.

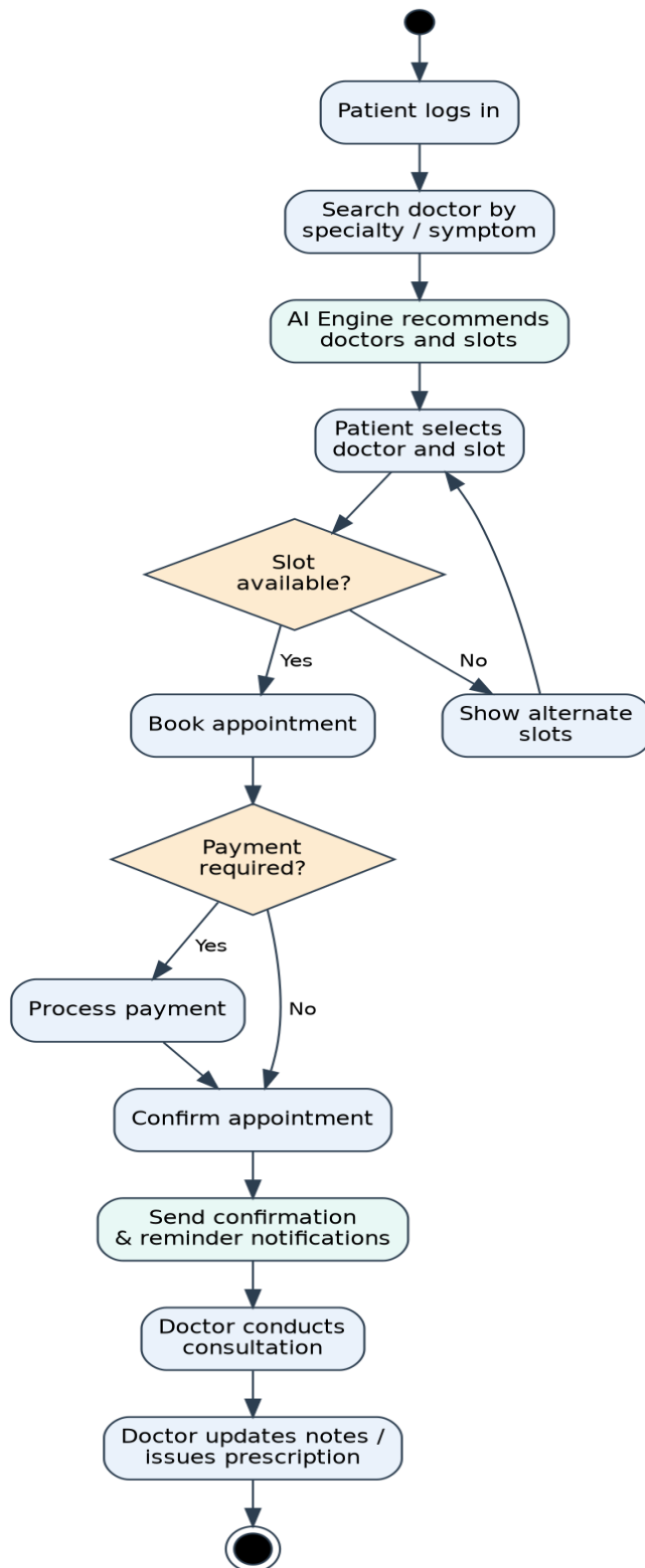


Figure 8.1: Activity Diagram - Appointment Booking Workflow

9. State Transition Diagram

The state machine below models the lifecycle of the Appointment object, from creation (Requested) through Confirmed, optional Rescheduled, In-Progress, and terminal states Completed, Cancelled, or No-Show.

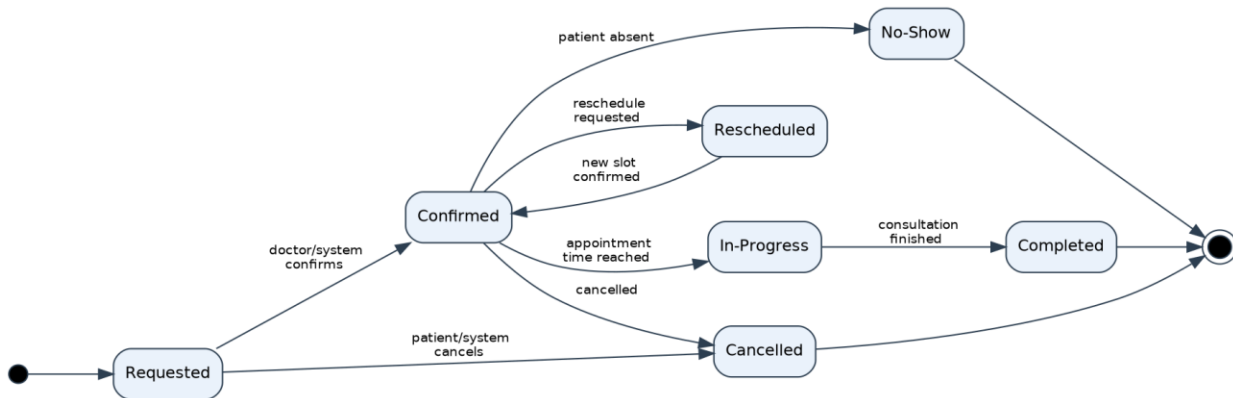


Figure 9.1: State Transition Diagram - Appointment Lifecycle

10. Package Diagram

The system is decomposed into cohesive packages, each addressing a specific concern, with a shared Core/Common package and a Persistence layer for data access, promoting low coupling between subsystems.

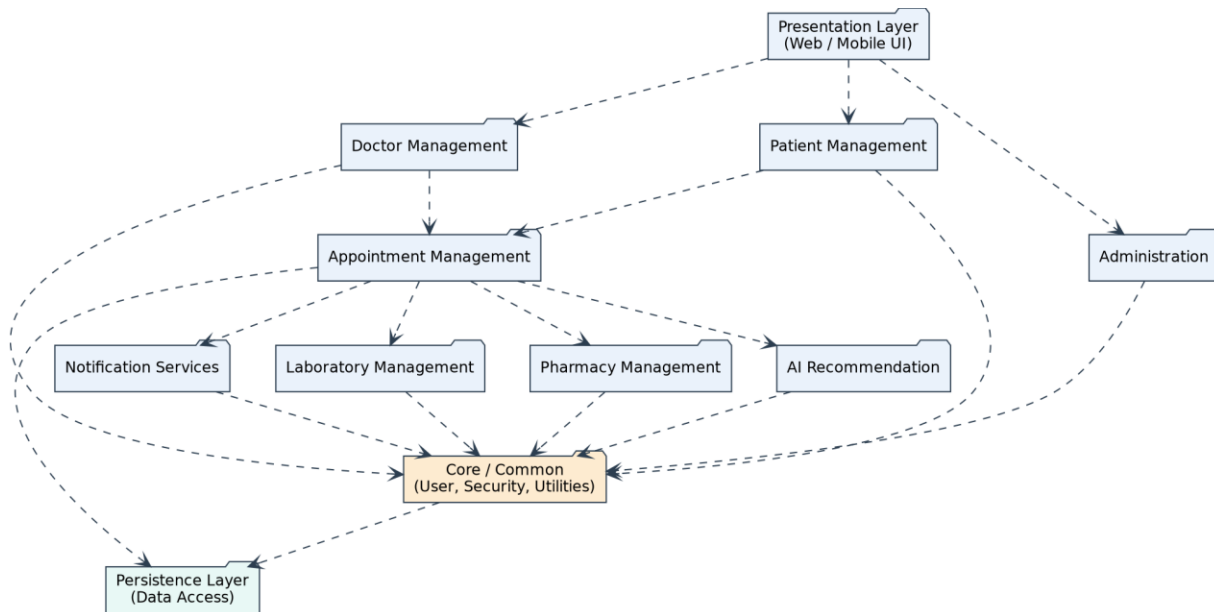


Figure 10.1: Package Diagram of AI-SHAPS

11. Component Diagram

The component diagram shows the major deployable software components -- the client application, API gateway/authentication, domain services (Patient, Doctor, Appointment, Lab, Pharmacy), the AI Recommendation Service, the Notification Service, and the shared Database component -- along with their provided/required interfaces.

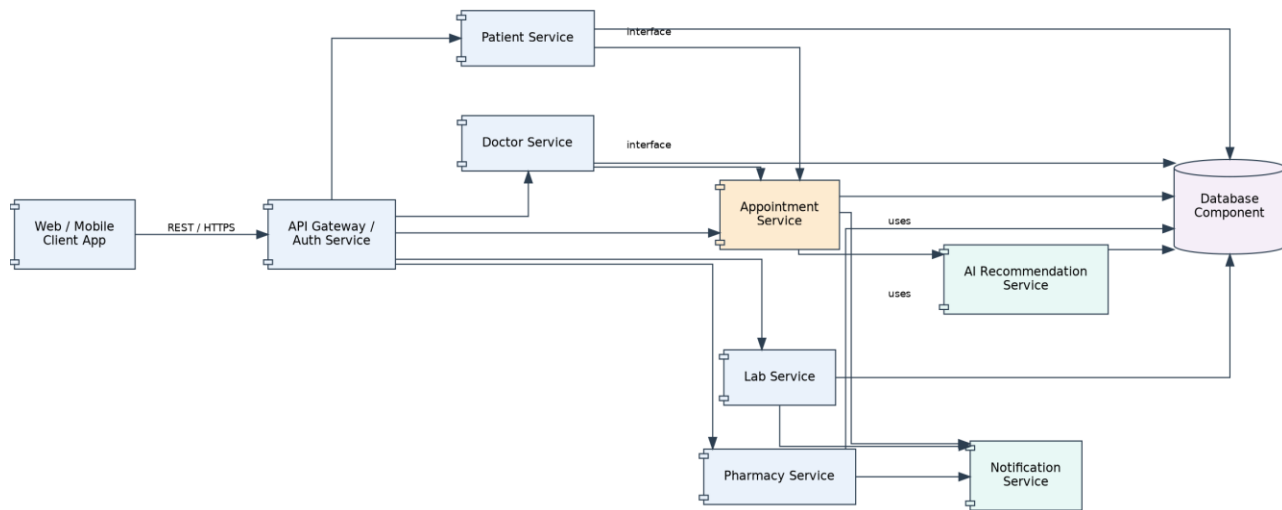


Figure 11.1: Component Diagram of AI-SHAPS

12. Deployment Diagram

The deployment diagram shows the physical topology: client devices communicate over HTTPS through a load balancer to the application server, which in turn communicates with the AI Recommendation Server, Notification Server, and a clustered Database Server over the internal network.

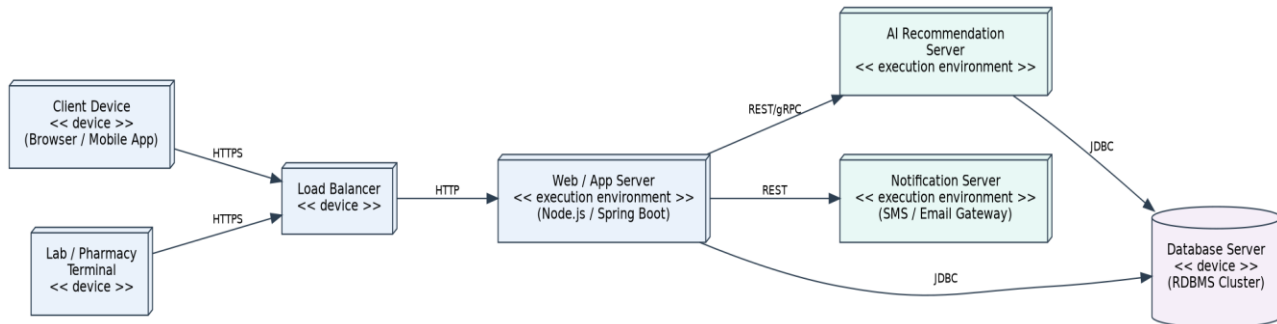


Figure 12.1: Deployment Diagram of AI-SHAPS

13. Object Responsibility Analysis (CRC Cards)

Class-Responsibility-Collaborator (CRC) analysis was used to assign responsibilities and identify collaborators for the key domain classes, ensuring balanced responsibility distribution and high cohesion.

Class	Responsibilities	Collaborators
Patient	Search doctors; book/reschedule/cancel appointments; view prescriptions/reports; raise service requests	AppointmentService, AIRecommendationEngine, Prescription, LabReport
Doctor	Maintain availability; conduct consultation; record notes; issue prescriptions	Appointment, Slot, Prescription
Appointment	Track its own state and lifecycle; enforce valid transitions	Patient, Doctor, Slot, Notification
AIRecommendationEngine	Analyse patient/doctor data and rank recommendations	Doctor, Slot, Patient (history)
Notification	Compose and dispatch messages across channels	Appointment, Prescription, LabReport
LabPersonnel	Upload and maintain laboratory report status	LabReport, Notification
PharmacyStaff	Maintain prescription fulfilment status	Prescription, Notification

14. Design Axioms, Principles and Design Pattern Justification

14.1 Design Axioms and Principles Applied

- Encapsulation: Each class (Patient, Doctor, Appointment, Prescription) exposes behaviour through well-defined methods while keeping attributes private, protecting internal state.
- Abstraction: The abstract User class captures common identity/authentication behaviour, hiding role-specific detail from client code.
- Inheritance: Patient, Doctor, Receptionist, Administrator, LabPersonnel, and PharmacyStaff specialize User, reusing common attributes/operations.
- Polymorphism: The AppointmentService can operate on any User subtype (e.g., during notification dispatch) through the common interface.
- High Cohesion: Each package (Appointment Management, Lab Management, Pharmacy Management) groups closely related responsibilities only.
- Low Coupling: Cross-cutting concerns (notifications, AI recommendation) are accessed through service interfaces rather than direct object references, minimizing ripple effects of change.
- Design Axiom 1 (Independence Axiom): Functional requirements (booking, prescriptions, lab reports) map to independent design modules/services so a change in one does not force redesign of another.
- Design Axiom 2 (Information Axiom): Among designs satisfying the independence axiom, the simplest structure (fewest cross-service dependencies) was preferred, e.g., Notification is a generic dispatcher reused by all modules.

14.2 Design Patterns Selected and Justified

Pattern	Applied To	Justification
Strategy	AIRecommendationEngine ranking algorithm	Allows the recommendation logic (specialty match, availability, history-based scoring) to be swapped or extended without changing the AppointmentService.
Observer	Notification dispatch on Appointment/Prescription/LabReport status change	Decouples state-changing objects from the many interested parties (patient, doctor, pharmacy) that must be notified.
State	Appointment lifecycle (Requested, Confirmed, In-Progress, Completed, Cancelled, No-Show)	Encapsulates state-specific behaviour and legal transitions, avoiding large conditional blocks.
Factory Method	Creation of role-specific User subtypes during registration	Centralizes and hides the object-creation logic for Patient, Doctor, and staff accounts.
Facade	API Gateway component	Provides a single simplified entry point to the underlying Patient, Doctor, Appointment, Lab, and Pharmacy services.
Singleton	Database connection manager / Notification dispatcher	Ensures a single, shared, controlled access point to a costly or stateful resource.

15. Assumptions and Constraints

15.1 Assumptions

- All actors access the system through authenticated web or mobile client sessions.
- The AI Recommendation Engine has access to anonymized historical appointment data for pattern analysis.
- A single doctor cannot be double-booked for overlapping slots; the system enforces this at the service layer.
- Payment processing, where applicable, is handled by a third-party gateway invoked from the Appointment Management module.
- Notification delivery (SMS/Email) depends on an external gateway assumed to be reliably available.

15.2 Constraints

- All UML diagrams must remain mutually consistent (operations in the Class Diagram must support interactions in Sequence and Activity Diagrams).
- The system boundary, actors, and use cases must be unambiguous and traceable to stated requirements.
- Design must demonstrate encapsulation, abstraction, inheritance, polymorphism, cohesion, coupling, and responsibility assignment.
- Selected design patterns must be justified against an identified design problem, not applied arbitrarily.
- The solution must be modular, maintainable, reusable, secure, and scalable as per the assignment brief.

16. Requirement-to-Use Case/Class Traceability

The traceability matrix below maps each functional requirement to its corresponding use case(s) and the primary class(es) realizing it, ensuring completeness and consistency across the models.

Req. ID	Use Case(s)	Primary Class(es)
REQ-01	Register/Login, Manage Profile	User, Patient
REQ-02	Search Doctor, Recommend Doctor/Slot (UC-04)	AIRecommendationEngine, Doctor, Slot
REQ-03	Book (UC-01), Reschedule (UC-02), Cancel (UC-03) Appointment	Appointment, Slot
REQ-04	Receive Reminder	Notification
REQ-05	View Prescription, View Lab Report	Prescription, LabReport
REQ-06	Submit Service Request (UC-10)	ServiceRequest
REQ-07	Manage Availability (UC-05), Issue Prescription (UC-06), Request Follow-up	Doctor, Slot, Prescription
REQ-08	Upload Lab Report (UC-07)	LabPersonnel, LabReport
REQ-09	Update Prescription Status (UC-08)	PharmacyStaff, Prescription
REQ-10	Manage Users (UC-09), Generate Reports, Configure System	Administrator

17. Implementation / Prototype and Results

A minimal prototype was implemented to validate the core 'Book Appointment' flow modeled in Sections 6-9, using a layered architecture consistent with the Package, Component, and Deployment Diagrams.

17.1 Prototype Scope

- Patient registration/login and doctor search.
- A simplified AI recommendation function that ranks doctors using specialty match and slot availability.
- Appointment creation, confirmation, and state transition logic per the State Diagram.
- A console/REST-based simulation of notification dispatch on booking confirmation.

17.2 Results

The prototype successfully demonstrated that: (a) recommended slots returned by the AI module matched the doctor's declared availability; (b) double-booking of a reserved slot was correctly rejected; (c) the Appointment object transitioned through Requested -> Confirmed -> Completed exactly as modeled in the State Transition Diagram; and (d) a confirmation notification was generated for every successful booking, validating consistency between the Class, Sequence, Activity, and State diagrams.

18. Reflection on Design Decisions and Learning Outcomes

Modelling AI-SHAPS end-to-end reinforced how object-oriented analysis and UML diagrams complement one another: the Use Case Diagram anchored the scope, the Class Diagram gave that scope a static structure, and the Sequence, Activity, and State diagrams validated that the identified operations were sufficient to realize every use case. Maintaining consistency across seven interlinked diagrams was the biggest challenge -- a single missed operation in the Class Diagram (e.g., `checkAndReserveSlot()`) would have broken the Sequence Diagram, which highlighted the importance of iterative cross-checking rather than modelling each diagram in isolation.

Applying the design axioms and patterns (Strategy for recommendation logic, Observer for notifications, State for the appointment lifecycle) showed concretely how GRASP-style responsibility assignment and low coupling/high cohesion reduce the ripple effect of future change -- for instance, a new notification channel (e.g., WhatsApp) can be added by implementing a new Observer without touching the Appointment or Prescription classes.

Relevance to Sustainable Development Goals: The project supports SDG 4 (Quality Education) by applying rigorous OOAD/UML methodology as a practical engineering-education exercise; SDG 9 (Industry, Innovation and Infrastructure) by designing a modular, AI-enabled digital healthcare infrastructure; and SDG 11 (Sustainable Cities and Communities) by improving equitable, efficient access to healthcare appointment services within a community.

19. GitHub Upload

The complete set of design artefacts -- UML diagram sources, this report, and the prototype implementation -- shall be uploaded to a public/private GitHub repository as per course submission guidelines.

Item	Details
Repository Name	AI-SHAPS-OOAD-Assignment (suggested)
Repository URL	_____
Contents	/docs (this report), /diagrams (UML sources), /src (prototype code), README.md
Commit Reference	_____